UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 7: DRAM Bandwidth and other Performance Considerations

Volodymyr Kindratenko

2/10/2026

Course Reminders

- Lab 3 is due this Friday
  - Make sure to implement all required code, including host memory allocation/deallocation.
  - Do not forget to push your work to github!
  - Submit your Lab 3 quiz only after you are done with the code.
- Getting help from our course chatbot; try asking these questions:
  - How do I allocate device memory?
  - How do I copy memory from host to device?
  - How do I call a GPU kernel?
  - Show an example of declaring a shared memory array in CUDA kernel.
  - Explain vector addition kernel code provided in the lectures.
  - Explain following code line by line in detail: “code goes here”
  - I am preparing for the exam, please help me to review the course materials from lectures 2 and 3. Ask me questions about it. Ask one question at a time, wait for my answer, and then check it for correctness.

Today's Objectives

- To understand the organization of memory based on dynamic RAM (DRAM)
- To understand the use of burst mode and multiple banks (both sources of parallelism) to increase DRAM performance (data rate)
- To understand memory access coalescing, which connects GPU kernel performance to DRAM organization
- To learn various kernel optimization techniques and tradeoff between them

Performance Optimizations

- Performance optimizations covered so far
  - Tuning resource usage to maximize occupancy to hide latency in cores
    - Threads per block, shared memory per block, registers per thread
    - Occupancy calculation
  - Reducing control divergence to increase SIMD efficiency
  - Shared memory and register tiling to reduce memory traffic

Performance Optimizations

- More optimizations to be covered today
  - Memory coalescing
  - Maximizing occupancy (again) to hide memory latency
  - Thread coarsening
  - Loop unrolling
  - Double-buffering

- Optimizations to be covered later
  - Privatization
  - Warp-level primitives

Review: Parallel Computing Challenges

- Global Memory Bandwidth
  - Ideal vs. Reality

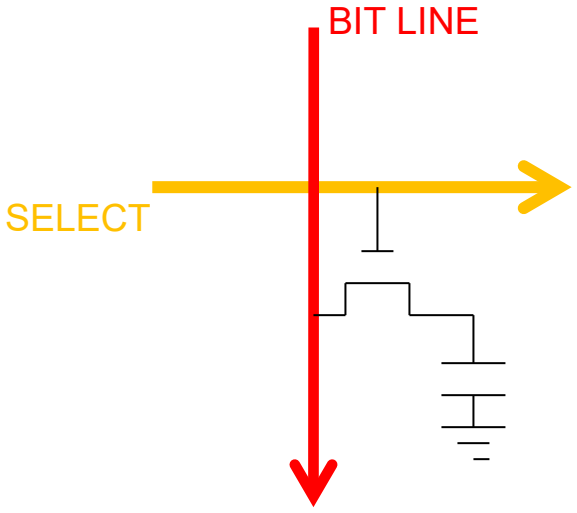
Ideal

Reality

- Conflicting Data Accesses Cause Serialization and Delays
  - Massively parallel execution cannot afford serialization
  - Contentions in accessing critical data causes serialization

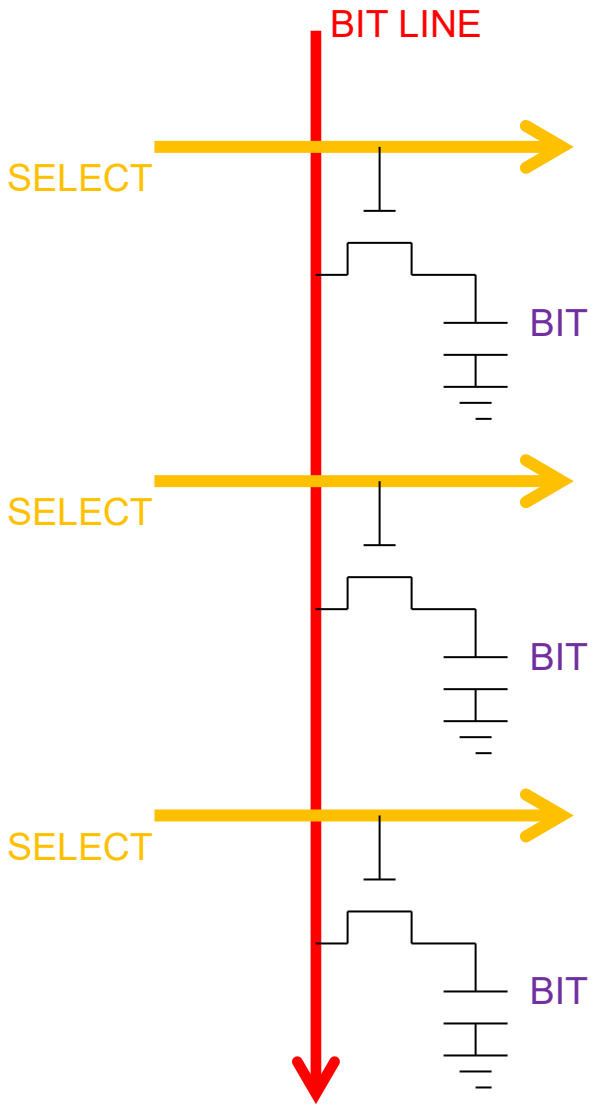
Most Large Memories Use DRAM

- **Random Access Memory (RAM)**:
  same time needed to read/write any address
- **Dynamic RAM (DRAM):**
  - **bit** stored on a capacitor
  - connected via transistor to **bit line** for read/write
  - **bits disappear** after a while (around 50 msec, due to tiny leakage currents through transistor), **and must be rewritten** (hence dynamic)



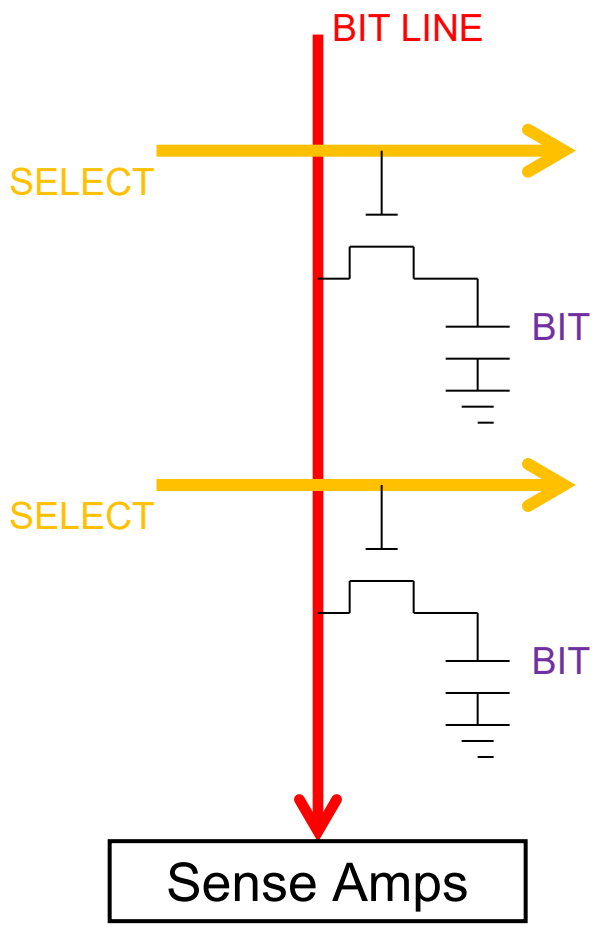
Many Cells (Bits) per Bit Line

- About **1,000 cells** connect to **each BIT LINE**
- **Connection**/disconnection **depends** on **SELECT** line
- Some **address bits** decoded to **connect exactly one cell to** the **BIT LINE**
- A DRAM bank consists of a 2D array of DRAM cells activated one row at a time, and read at the column



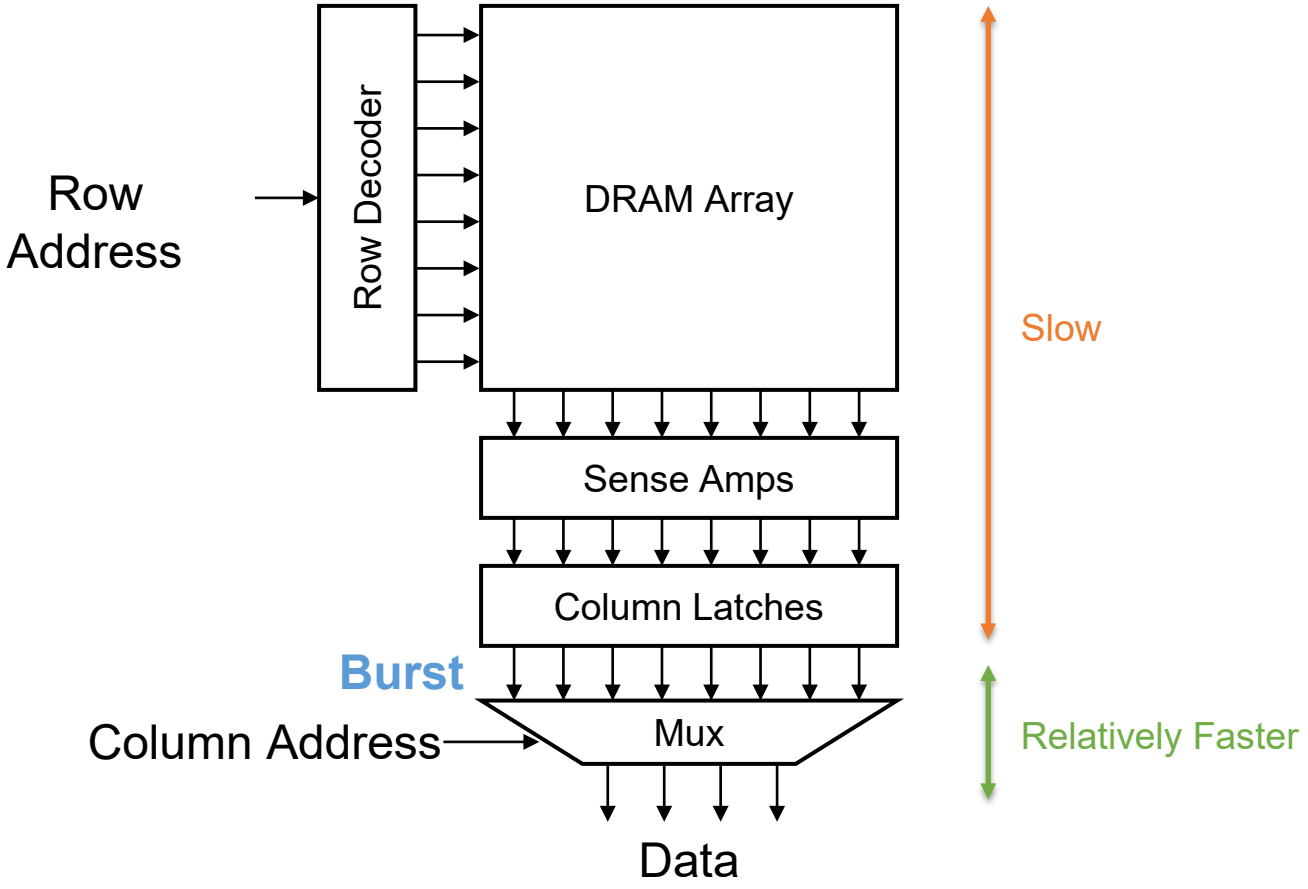
DRAM is Slow But Dense

- Capacitance…
  - tiny for the **BIT**, but
  - huge for the **BIT LINE**
- Use an amplifier for higher speed!
- Still **slow**…
- But only need **1 transistor per bit**



DRAM Bank Organization

- A DRAM bank consists of a 2D array of DRAM cells activated one row at a time, and read at the column
- **SELECT** lines connect to about 1,000 bit lines
- Core DRAM array has about O(1M) bits
- Use more address bits to choose bit line(s)



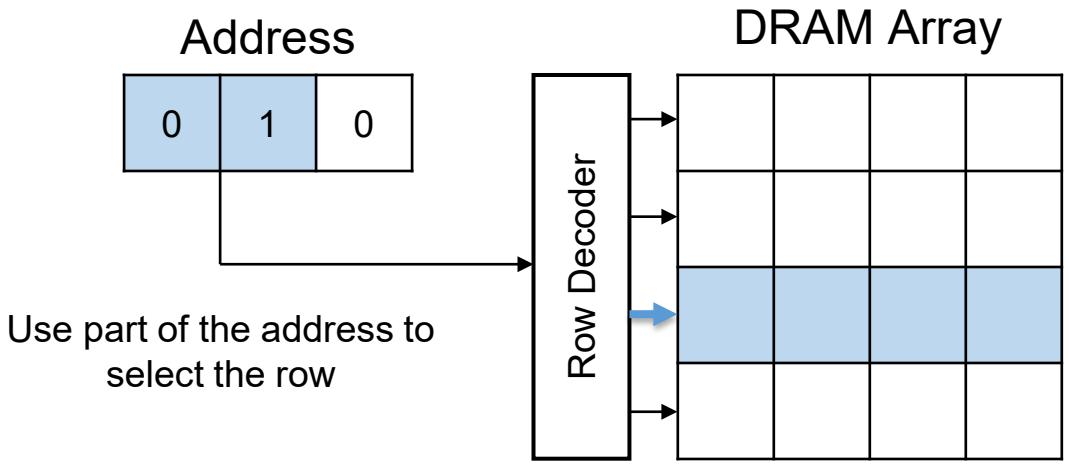
A Very Small (8x2 bit) DRAM Bank Example

Address

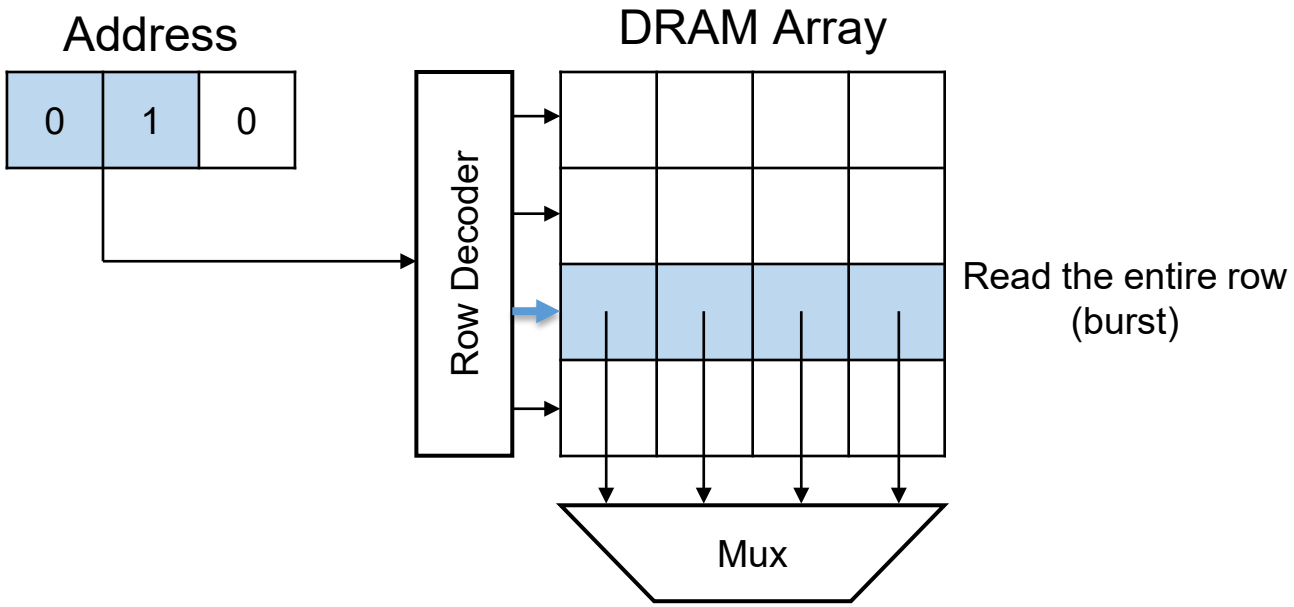
0	1	0

DRAM Array

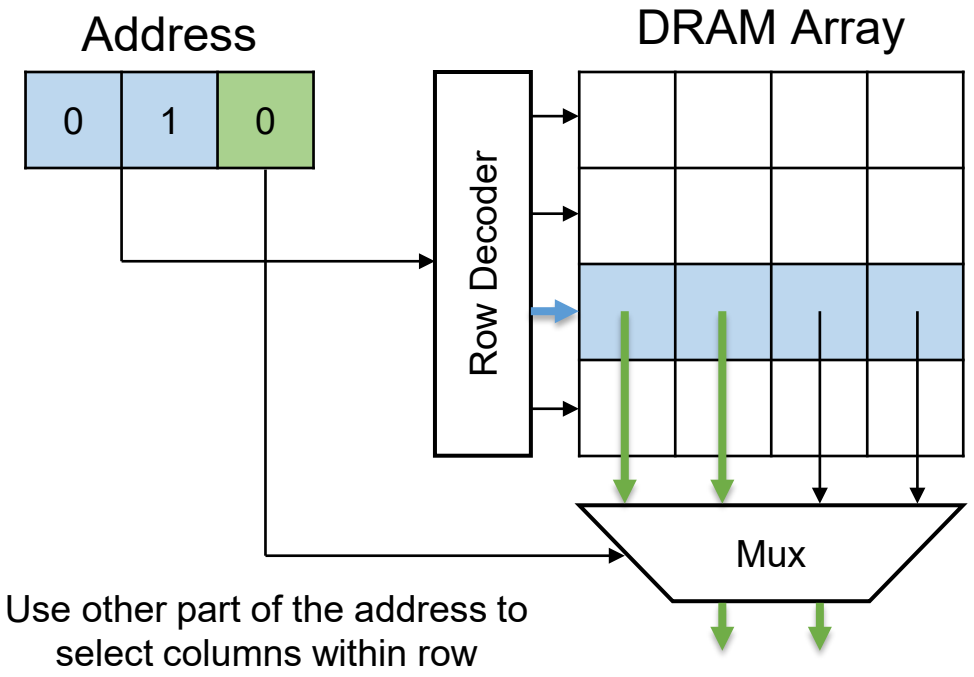
A Very Small (8x2 bit) DRAM Bank Example



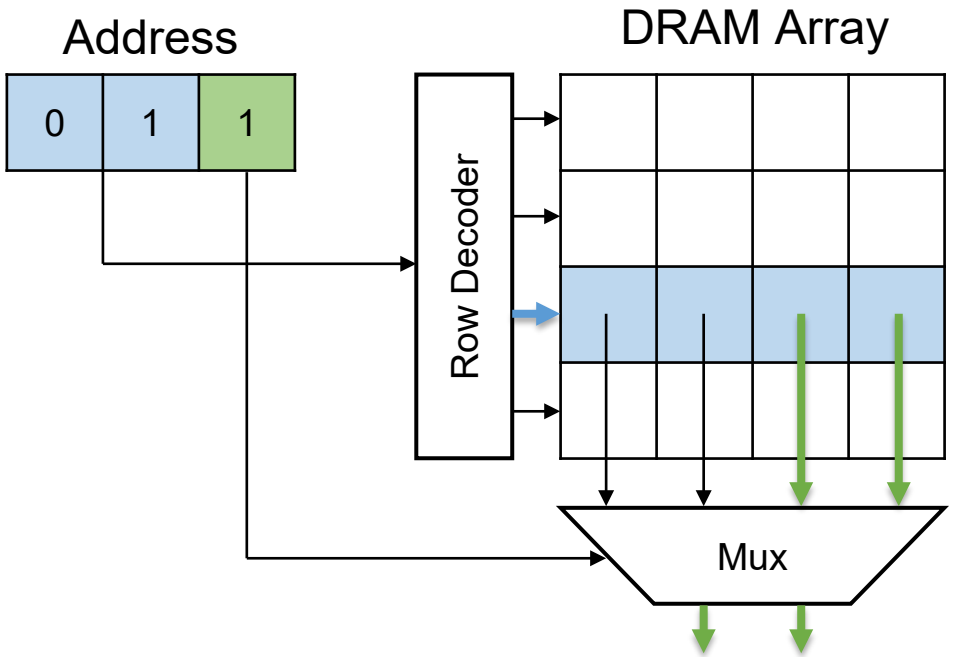
A Very Small (8x2 bit) DRAM Bank Example



A Very Small (8x2 bit) DRAM Bank Example



A Very Small (8x2 bit) DRAM Bank Example



If other access from the same burst, no need to read the row again

DRAM Bursting for the 8x2 Bank

- Accessing data in different bursts
  - Need to access the array again

- Accessing data in the same burst
  - No need to access the array again, just the multiplexer

- Accessing data in the same burst is faster than accessing data in different bursts

Memory Coalescing

- When threads in the same warp access consecutive memory locations in the same burst, the accesses can be combined and served by one burst
  - One DRAM transaction is needed
  - Known as **memory coalescing**

- If threads in the same warp access locations not in the same burst, accesses cannot be combined
  - Multiple transactions are needed
  - Takes longer to service data to the warp
  - Sometimes called **memory divergence**

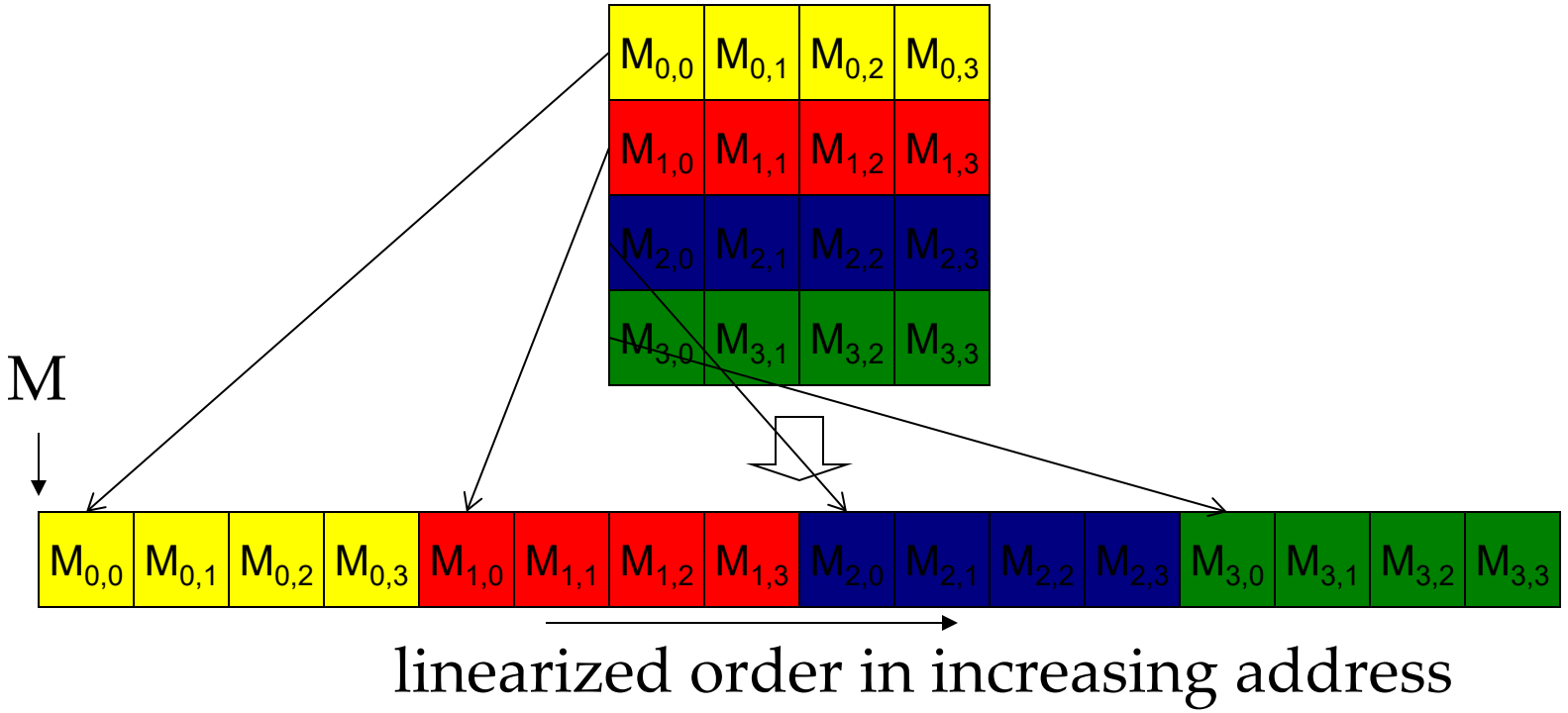
Memory Coalescing Examples

- Vector addition

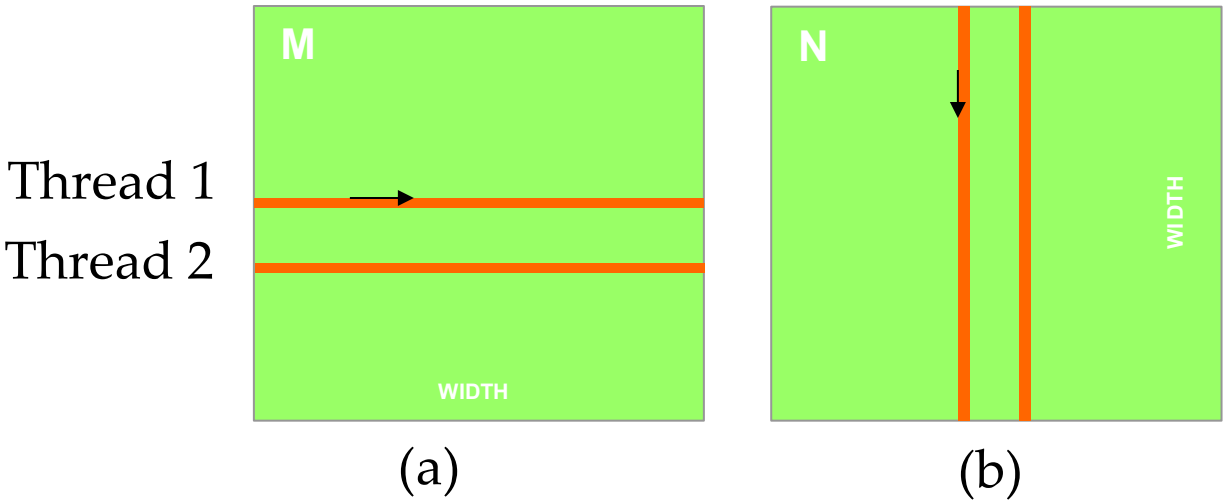
```
int i = blockDim.x*blockIdx.x + threadIdx.x;
z[i] = x[i] + y[i];
```

  - Accesses to `x`, `y`, and `z` are coalesced
    - e.g., threads 0 to 31 access elements 0 to 31, resulting in one memory transaction to service warp 0

Review: Placing a 2D C Array Into Linear Memory Space



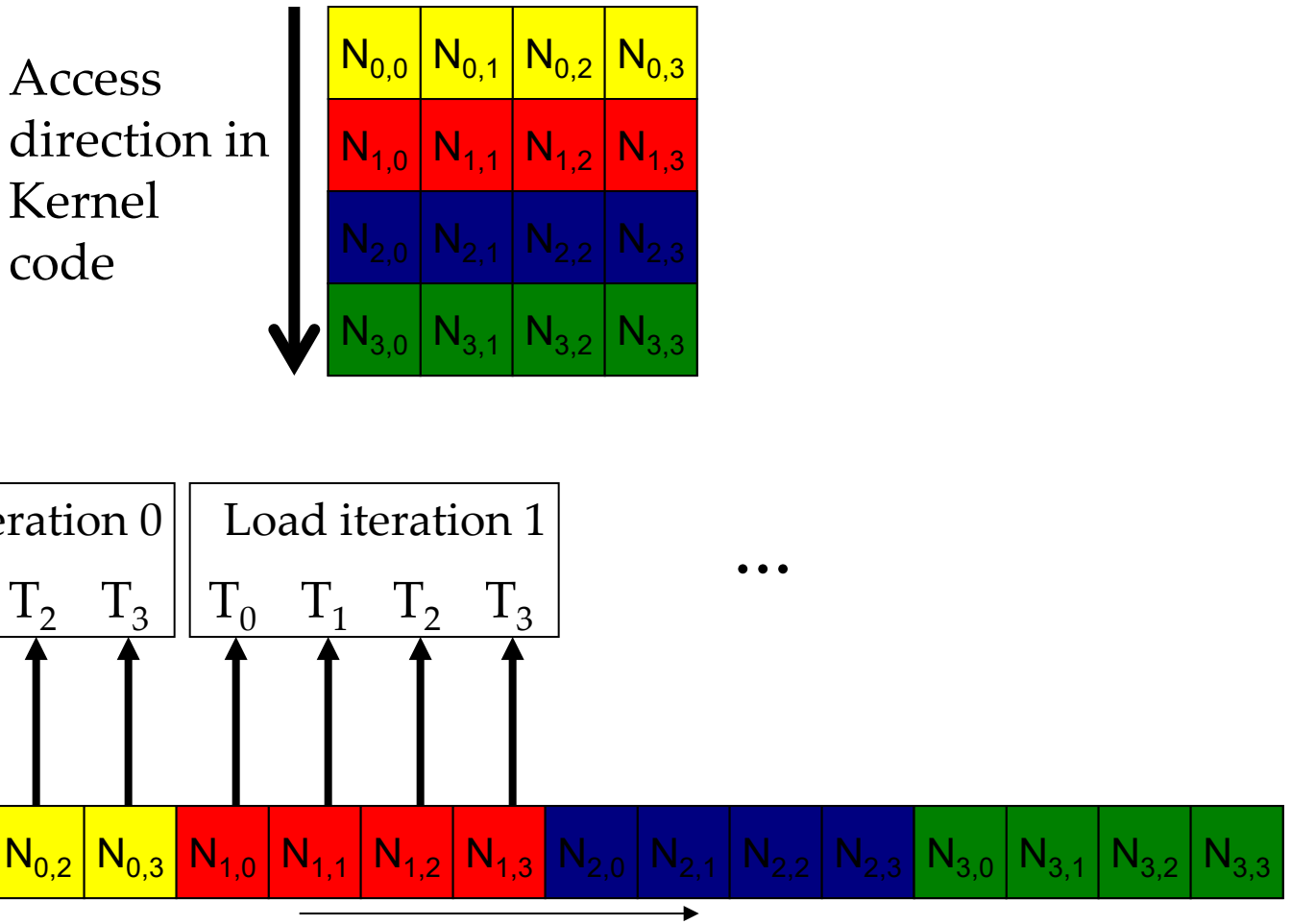
Two Access Patterns



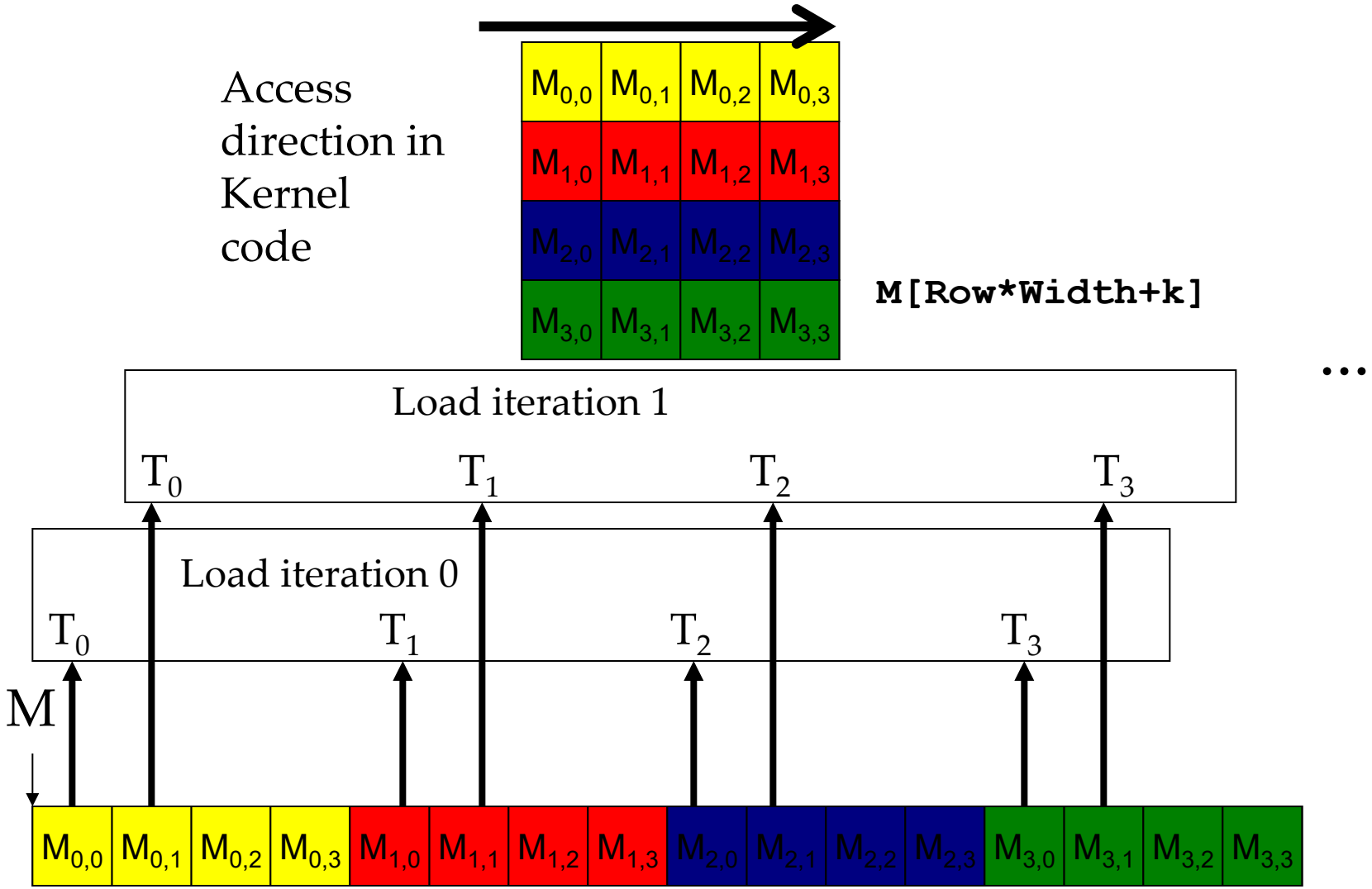
`M[Row*Width+k]` `N[k*Width+Col]`

k is loop counter in the inner product loop of the kernel code

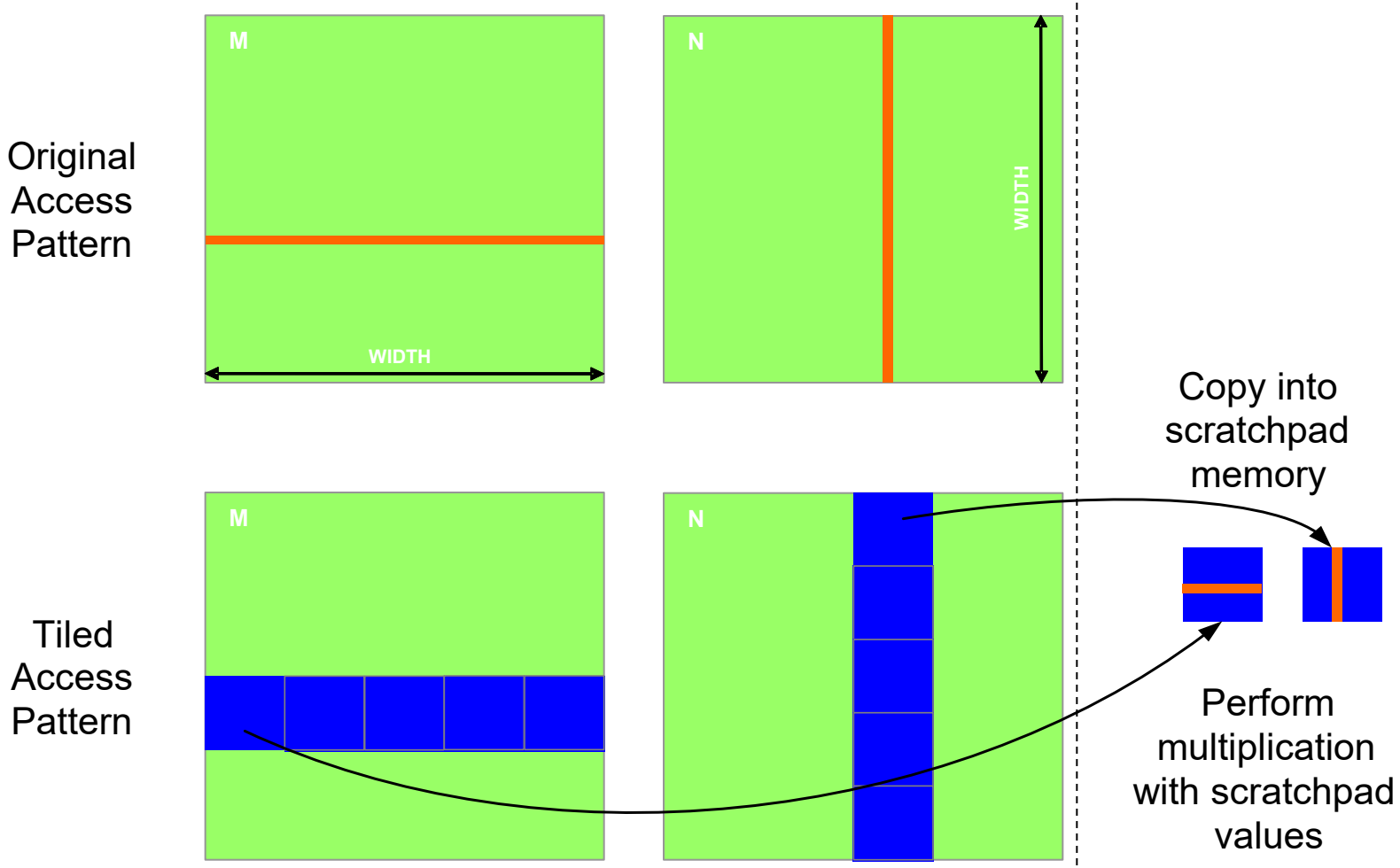
N Accesses Are Coalesced



M Accesses Are Not Coalesced



Use of Shared Memory Enables Coalescing



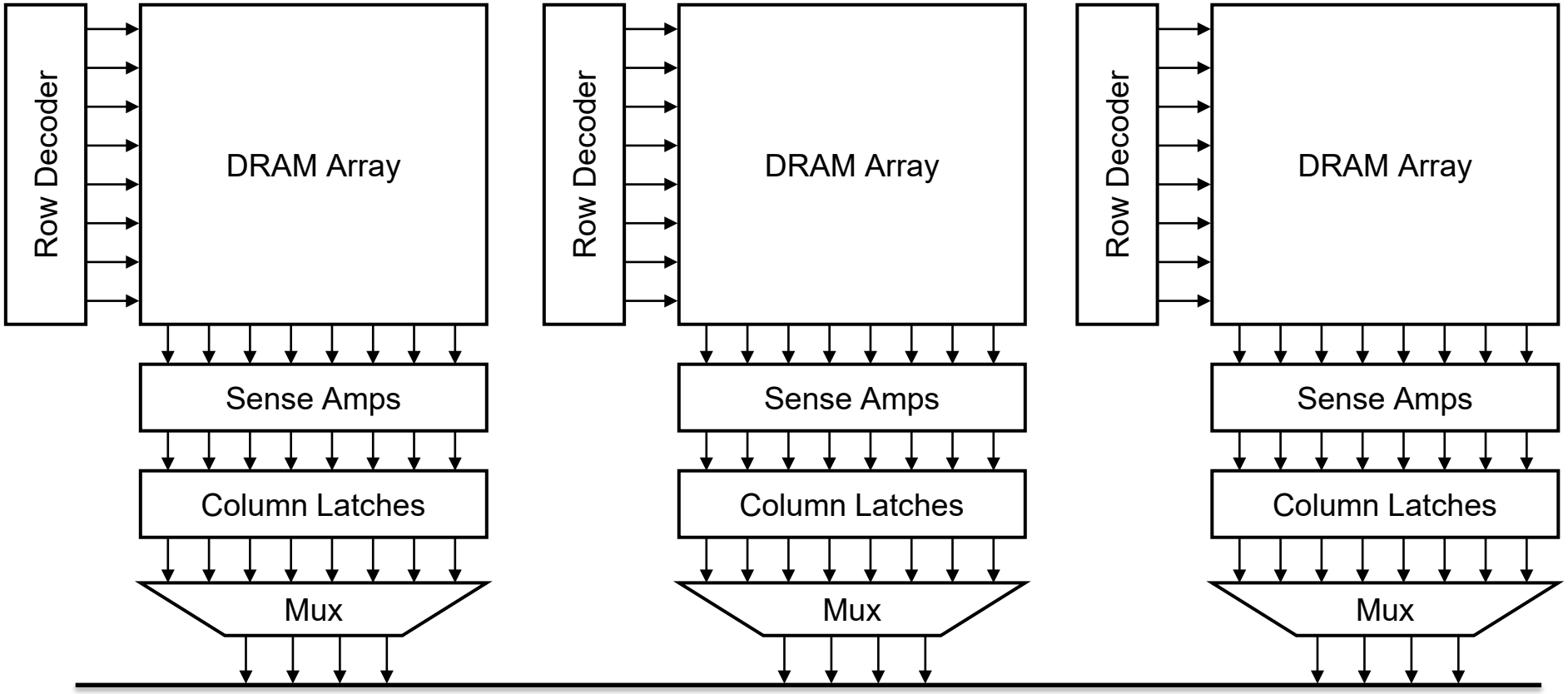
Memory Coalescing Examples

- Tiled matrix-matrix multiplication

```
unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;
For (unsigned int tile = 0; tile < N/TILE_DIM; ++tile) {
    M_s[threadIdx.y][threadIdx.x] = M[row*N + tile*TILE_DIM + threadIdx.x];
    N_s[threadIdx.y][threadIdx.x] = N[(tile*TILE_DIM + threadIdx.y)*N + col];
    ...
}
```

- Accesses to M and N are coalesced
  - e.g., threads 0 to 31 access elements 0 to 31 of M on the first iteration, resulting in one memory transaction to service warp 0
    - Even better than without tiling where all threads accessed one element, underutilizing the burst
  - e.g., threads 0 to 31 access elements 0 to 31 of N on the first iteration, resulting in one memory transaction to service warp 0

Multiple DRAM Banks



Latency Hiding with Multiple Banks

- With one bank, time still wasted in between bursts

- Latency can be hidden by having multiple banks

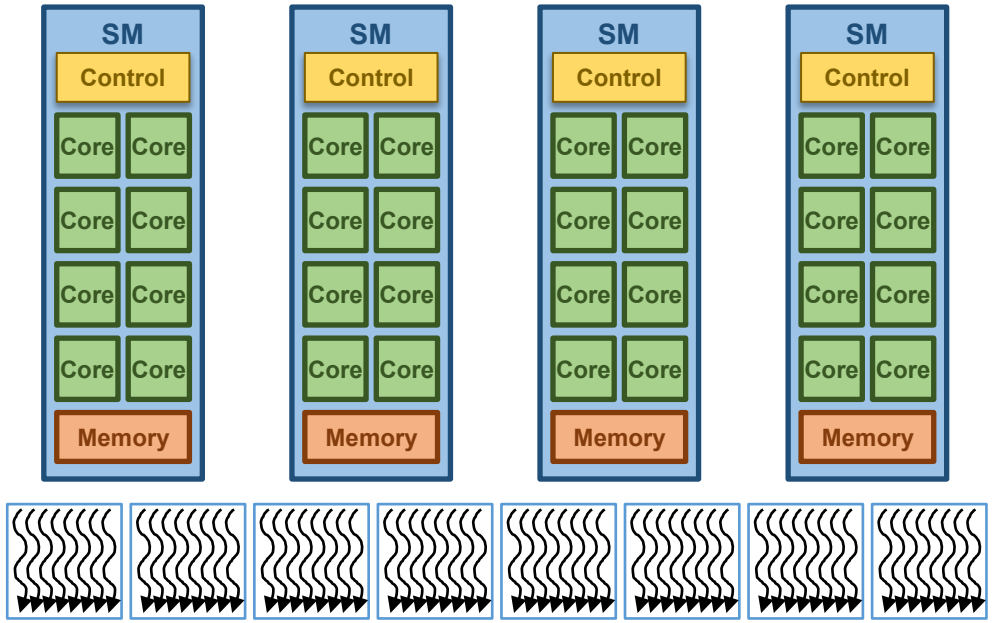
- Need many threads to simultaneously access memory to keep all banks busy
  - Achieved with having high occupancy in SMs
    - Similar idea to hiding pipeline latency in the core

Fine-Grain Thread Granularity

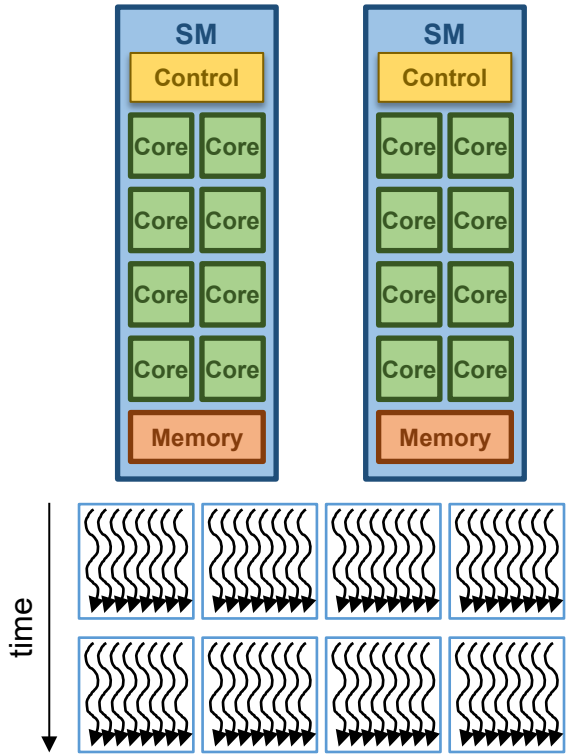
- So far, parallelization approaches made threads as *fine-grain* as possible
  - Assign smallest possible unit of parallelizable work per thread
    - e.g., one vector element per thread in vector addition
    - e.g., one output pixel per thread in RGB to gray and in blur
    - e.g., one output matrix element per thread in matrix-matrix multiplication

Fine-Grain Thread Granularity

If the GPU has a lot of resources, they will be maximally utilized



If more threads are provided than the GPU can support, the hardware can serialize the work with low overhead



But if an overhead was incurred to parallelize the work across more threads, that overhead would have been incurred unnecessarily ☹

Fine-Grain Thread Granularity

- Advantage: provide hardware with as many threads as possible to fully utilize resources
  - If more threads are provided than the GPU can support, the hardware can serialize the work with low overhead
  - If future GPUs come out with more resources, more parallelism can be extracted without code being rewritten
    - Recall: *transparent scalability*
- Disadvantage: if there is an overhead for parallelizing work across more threads, that overhead is maximized
  - Okay if threads actually run in parallel
  - Suboptimal if threads are getting serialized by the hardware

Thread Coarsening

- **Thread coarsening** is an optimization were a thread is assigned multiple units of parallelizable work
  - i.e., a thread is made more *coarse-grain*

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
foo(i);
```

↓ Thread coarsening

```
unsigned int iStart = 4*(blockIdx.x*blockDim.x + threadIdx.x);
for(unsigned int c = 0; c < 4; ++c) {
    unsigned int i = iStart + c;
    foo(i);
}
```

Thread Coarsening

- Advantages
  - Reduces the overhead incurred for parallelization
    - Could be redundant memory accesses
    - Could be redundant computations
    - Could be synchronization overhead or control divergence
    - We will see many examples throughout the course

- Disadvantages
  - More resources per thread which may affect occupancy
  - Underutilizes resources if *coarsening factor* is too high
    - Need to retune coarsening factor for each device

Loop Unrolling

- **Loop unrolling** transforms a loop by replicating the body of the loop by some factor and reducing the number of loop iterations by the same factor

- Loop unrolling reduces stalls in two ways:
  - Fewer loop iterations implies fewer branches
    - Branches have long-latency in the absence of branch prediction
  - Exposes more independent instructions for *instruction scheduling*

```
for(unsigned int i = 0; i < 16; ++i) {
    A(i)
    B(i)
}
```

Loop unrolling

```
for(unsigned int i = 0; i < 16; i += 4) {
    A(i)
    B(i)
    A(i + 1)
    B(i + 1)
    A(i + 2)
    B(i + 2)
    A(i + 3)
    B(i + 3)
}
```

Instruction Scheduling

- **Instruction scheduling** reorders instructions to reduce stalling by placing instructions that depend on each other farther away from each other

```
for(unsigned int i = 0; i < 16; i += 4) {
    A(i)
    B(i)
    A(i + 1)
    B(i + 1)
    A(i + 2)
    B(i + 2)
    A(i + 3)
    B(i + 3)
}
```

Instruction scheduling

```
for(unsigned int i = 0; i < 16; i += 4) {
    A(i)
    A(i + 1)
    A(i + 2)
    A(i + 3)
    B(i)
    B(i + 1)
    B(i + 2)
    B(i + 3)
}
```

Loop Unrolling and Instruction Scheduling in the Compiler

- In practice, loop unrolling and instruction scheduling are applied automatically by the compiler
  - Can be controlled with directives

```
#pragma unroll 4
for(unsigned int i = 0; i < 16; ++i) {
    A(i)
    B(i)
}
```

Loop Unrolling and Thread Coarsening

- A common source of loops that can be unrolled is thread coarsening
  - Added benefit of unrolling is enabling promoting local arrays to registers

```
int x;
foo(x);
```

Thread coarsening

```
int x[4];
for(unsigned int c = 0; c < 4; ++c) {
    foo(x[c]);
}
```

x placed in global memory instead of registers because needs to be accessible with a variable index

Constant coarsening factor allows the coarsening loop to be fully unrolled

Loop unrolling

```
int x[4];
foo(x[0]);
foo(x[1]);
foo(x[2]);
foo(x[3]);
```

x can be promoted to registers because all accesses to x have a constant index

Double Buffering

- **Double buffering** eliminates false dependences by using a different memory buffer for writing data than the memory buffer containing the data being read

```
for(unsigned int i = 0; i < N; ++i) {
    ... = buffer[anotherThreadID];
    __syncthreads();
    buffer[myThreadID] = ...;
    __syncthreads();
}
```

This synchronization enforces a **false dependence** (we only need to finish reading before others write because we are using the same buffer)

This synchronization enforces a **true dependence** (we must finish writing before others can read)

Double buffering

```
for(unsigned int i = 0; i < N; ++i) {
    ... = inBuffer[anotherThreadID];
    outBuffer[myThreadID] = ...;
    __syncthreads();
    swap(inBuffer, outBuffer);
}
```

No synchronization needed because writes to outBuffer to not affect the data in inBuffer

Checklist of Common Optimizations

Category	Optimization	Benefit to Compute Cores	Benefit to Memory	Strategies
Compute utilization	Occupancy tuning	More work to hide pipeline latency	More parallel memory accesses to hide DRAM latency	Tune the usage of SM resources such as threads per block, shared memory per block, and registers per thread
	Loop unrolling	Fewer branch instructions and more independent instruction sequences with fewer stalls	May enable promoting local arrays to registers to reduce global memory traffic	Performed automatically by the compiler Use loops with constant bounds where possible to facilitate the compiler's job
	Reducing control divergence	High SIMD efficiency (fewer idle cores during SIMD execution)	-	Rearrange the assignment of threads to work and/or data Rearranging the layout of the data
Memory utilization	Using coalesceable global memory accesses	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic and better utilization of bursts/cache-lines	Rearranging the mapping of threads to data Rearranging the layout of the data Transfer data between global memory and shared memory in a coalesceable manner and perform irregular accesses in shared memory (e.g., corner turning)
	Shared memory tiling	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic	Place data that is reused within a block in shared memory so that it is transferred between global memory and the SM only once
	Register tiling	Fewer pipeline stalls waiting for shared memory accesses	Less shared memory traffic	Place data that is reused within a warp or thread in registers so that it is transferred between shared memory and registers only once
	Privatization	Fewer pipeline stalls waiting for atomic updates	Less contention and serialization of atomic updates	Apply partial updates to a private copy of the data then update the public copy when done
Synchroni-zation latency	Warp-level primitives	Reduce block-wide barrier synchronizations	Less shared memory traffic	Perform operations requiring barrier synchronization at the warp-level, then consolidate warp-level results at the block-level
	Double buffering	Eliminates barriers that enforce false dependences	-	Eliminate false (write-after-read) dependences by using different buffers for the writes and the preceding reads
General	Thread coarsening	Depends on the overhead of parallelization	Depends on the overhead of parallelization	Assign multiple units of parallelism to each thread in order to reduce the overhead of parallelization when it is incurred unnecessarily

Trade-off Between Optimizations

- Maximizing occupancy
  - Maximizing occupancy hides pipeline latency, but threads may compete for resources (e.g., registers, shared memory, cache)
- Shared memory tiling
  - Using more shared memory enables more data reuse, but may limit occupancy
- Thread coarsening
  - Coarsening reduces parallelization overhead, but requires more resources per thread which may limit occupancy
- How do we know which optimization to apply?

Know Your Application's Bottleneck!

- The constraint that limits the performance of an application on a device is called a **bottleneck**
  - The bottleneck depends on the application as well as the device itself
- Optimizations trade one resource for another to alleviate the bottleneck
- Make sure to properly diagnose your application's bottleneck before applying optimizations
  - Otherwise, you may be optimizing for the wrong resource
- Know when to stop optimizing by comparing your application's performance to speed-of-light performance

Any More Questions?

Things to Read:

Textbook chapter 6

CUDA BPG: Memory Optimizations

Things to Do:

Submit Lab 3

Sign up for GPT-2 project, if you wish

Problem Solving

- Q: Which of the following two lines of a kernel code has better performance?

```
   int tx = blockIdx.x * blockDim.x + threadIdx.x;
   int ty = blockIdx.y * blockDim.y + threadIdx.y;
A) B[ty * Width + tx] = 2 * A[ty * Width + tx];
B) B[tx * Width + ty] = 2 * A[tx * Width + ty];
```

- A: Cannot tell. We do not know how the kernel was launched.

Problem Solving

- Q: Consider a DRAM system with a burst size of 512 bytes and a peak bandwidth of 240 GB/s. Assume a thread block size of 1024 and warp size of 32 and that A is a floating-point array in the global memory. What is the maximal memory data access throughput we can hope to achieve in the following access to A?

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
float temp = A[4*i] + A[4*i+1];
```

- A:
  - From a burst of 512 bytes, we will only use every other 8 bytes due to reading from [4*i] and [4*i+1] memory locations.
  - So, we can expect 120 GB/s.